Project Files Structure - 001

Ordoñez Silva, Yonel Jr.

Indice

Arquitectura Completa del Sistema SaaS Zootécnico **3**

VISIÓN GENERAL **3**
Diagrama C4 4
Diagrama de Despliegue 6

DOMAIN **6**
Diagrama C4 9
Responsabilidad 10
Entities/ 10
Product.cs 10
Animal.cs 10
MilkRecord.cs 10
ReproductionEvent.cs 11
AuditEntry.cs 11
ValueObjects/ 11
AnimalId.cs 11
Weight.cs 11
MilkQuantity.cs 12
Enums/ 12
AnimalStatus.cs 12
ReproductionType.cs 12
Rules/ 12
Exceptions/ 12
DomainException.cs 12
Services/ 13

APPLICATION **13**
Diagrama C4 15
Responsabilidad 15

Common/Gateway/ . . . 16
Context/ . . . 16
Caching/ . . . 16
Configuration/ . . . 16
Features/ . . . 16
Rules/ . . . 16
Identity/ . . . 16
Time/ . . . 16
Behaviors/ . . . 17
Modules/ . . . 17
Animals/ . . . 17
Interfaces/ . . . 17
UseCases/CreateAnimal/ . . . 17
Interactor . . . 18

INTERFACE ADAPTERS **18**
Diagrama C4 . . . 19
Responsabilidad . . . 19
Controllers/ . . . 19
DTOs/ . . . 20
Mappers/ . . . 20
Presenters/ . . . 20
Middleware/ . . . 20
TenantResolutionMiddleware . . . 20
Filters/ . . . 20

INFRASTRUCTURE **20**
Diagrama C4 . . . 24
Responsabilidad . . . 25
Persistence/ . . . 25
DbContext . . . 25
Repositories . . . 25
Tenant/ . . . 25
TenantContext . . . 25
TenantStore . . . 25
TenantDbContextFactory . . . 25
Configuration/ . . . 26
Features/ . . . 26
Rules/ . . . 26
Caching/ . . . 26
Context/ . . . 26
Identity/ . . . 26
Auditing/ . . . 27

External/ 27

FLUJO COMPLETO **27**

CONCLUSIÓN **27**

Arquitectura Completa del Sistema SaaS Zootécnico

Este documento describe en detalle **cada capa, componente, carpeta y archivo** de la arquitectura basada en Clean Architecture + enfoque modular + multi-tenant.

VISIÓN GENERAL

```
InterfaceAdapters → Application → Domain
                      ↑
                Infrastructure
```

- **Domain**: reglas del negocio
- **Application**: orquestación (casos de uso)
- **InterfaceAdapters**: entrada/salida (HTTP, UI)
- **Infrastructure**: implementación técnica

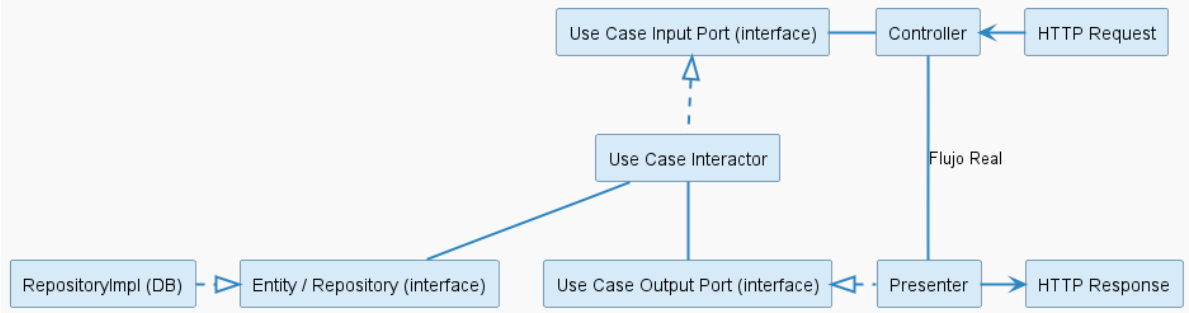

Figure 1: Flujo

Diagrama C4

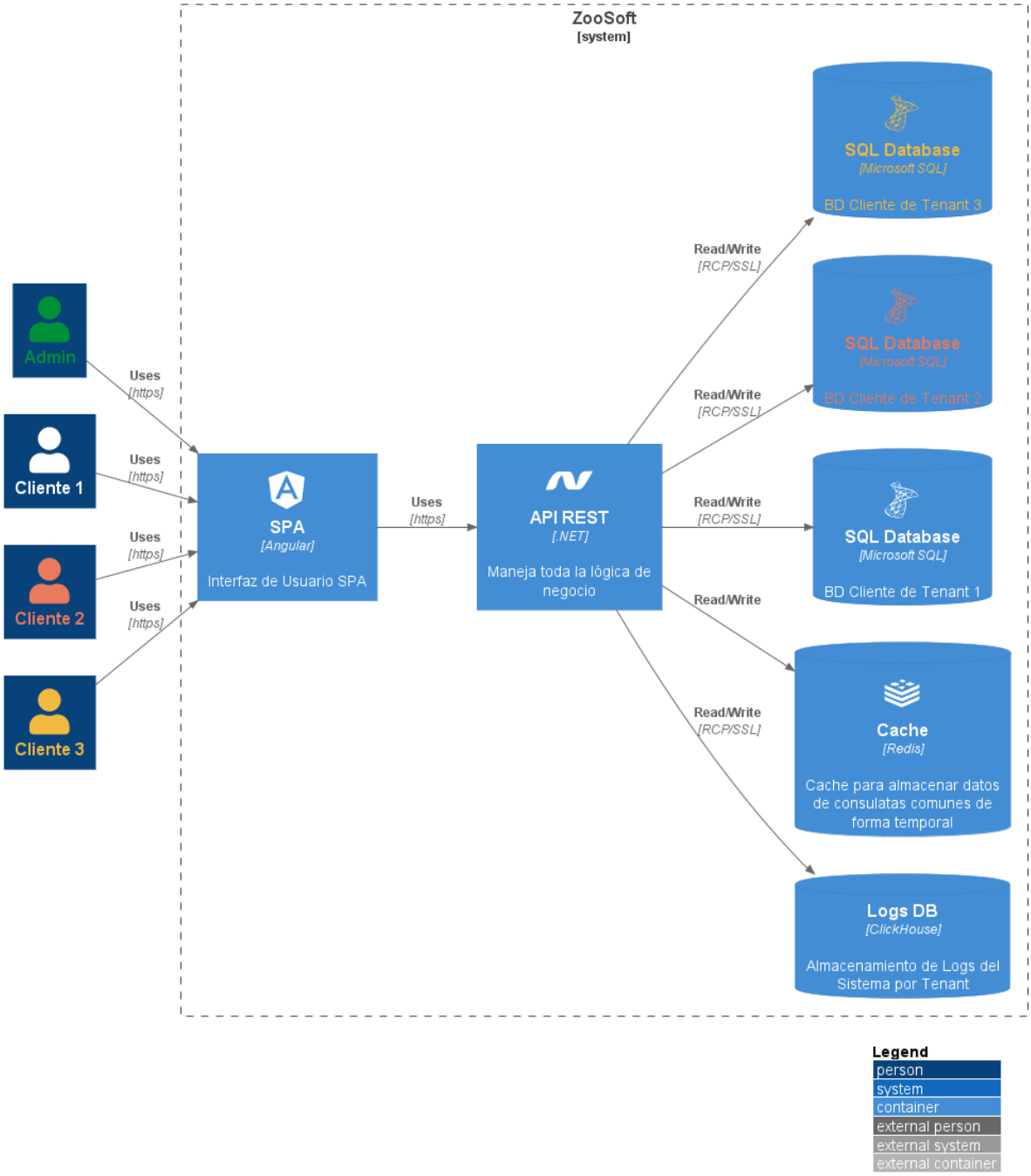

Figure 2: Diagrama C4 - General

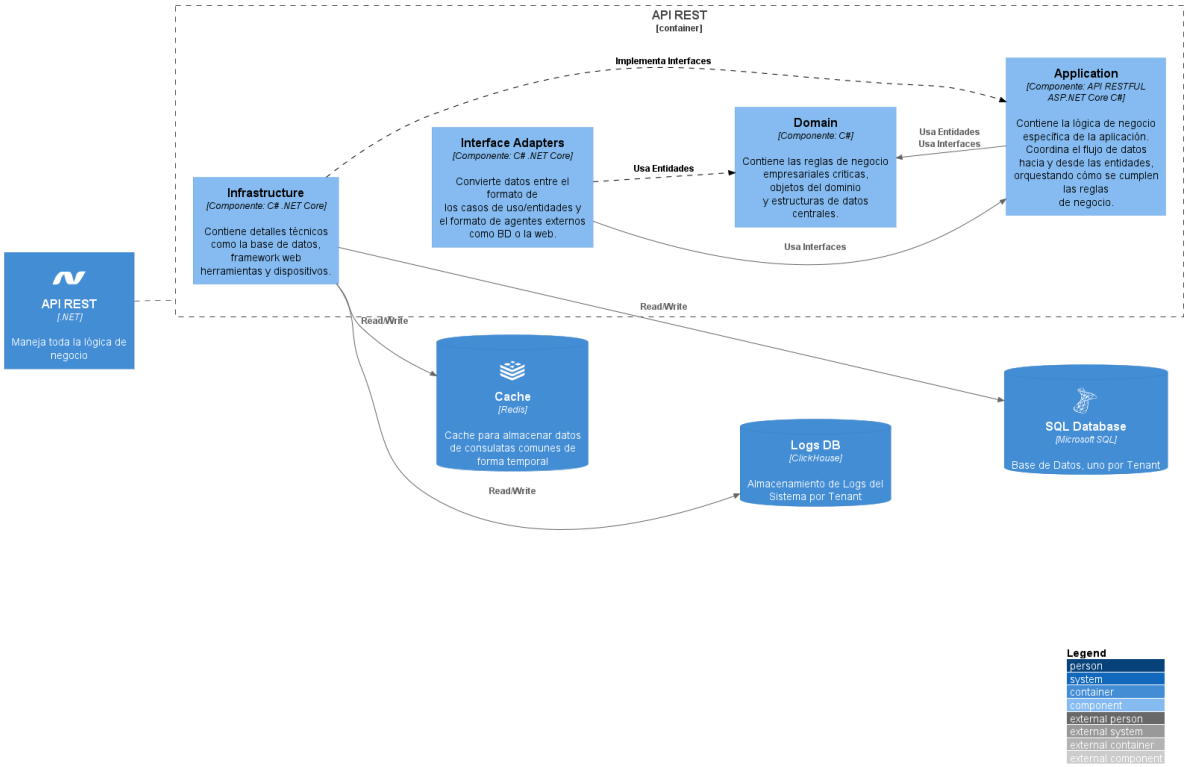

Figure 3: Diagrama C4 - Backend

Diagrama de Despliegue

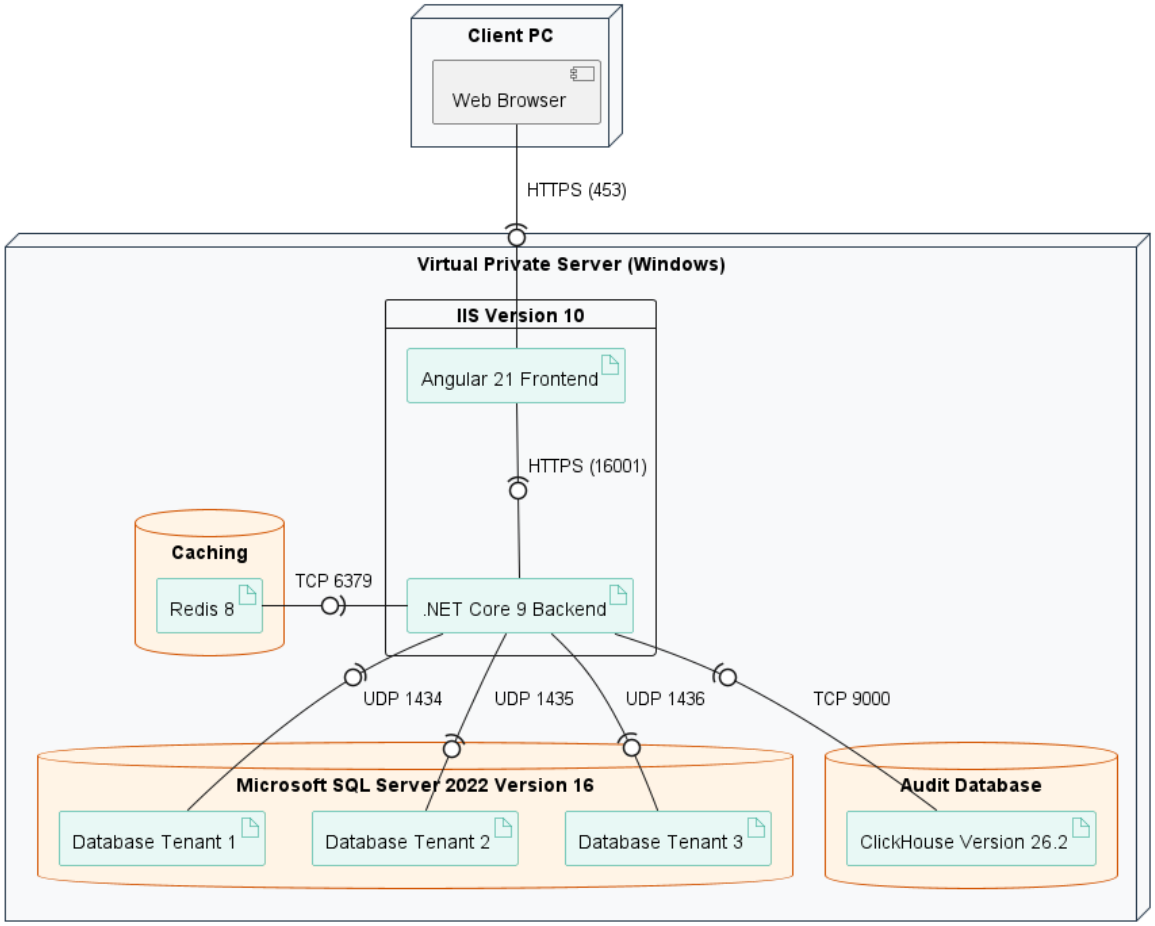

Figure 4: Diagrama de Despliegue

DOMAIN

```
Domain/
    Entities/
          Product.cs
          Animal.cs
```

```
        MilkRecord.cs
        ReproductionEvent.cs
        AuditEntry.cs

  ValueObjects/
        AnimalId.cs
        Weight.cs
        MilkQuantity.cs

  Enums/
        AnimalStatus.cs
        ReproductionType.cs

  Rules/
        BusinessRule.cs

  Exceptions/
        DomainException.cs

  Services/
         (opcional: lógica compleja de dominio)
```

Diagrama C4

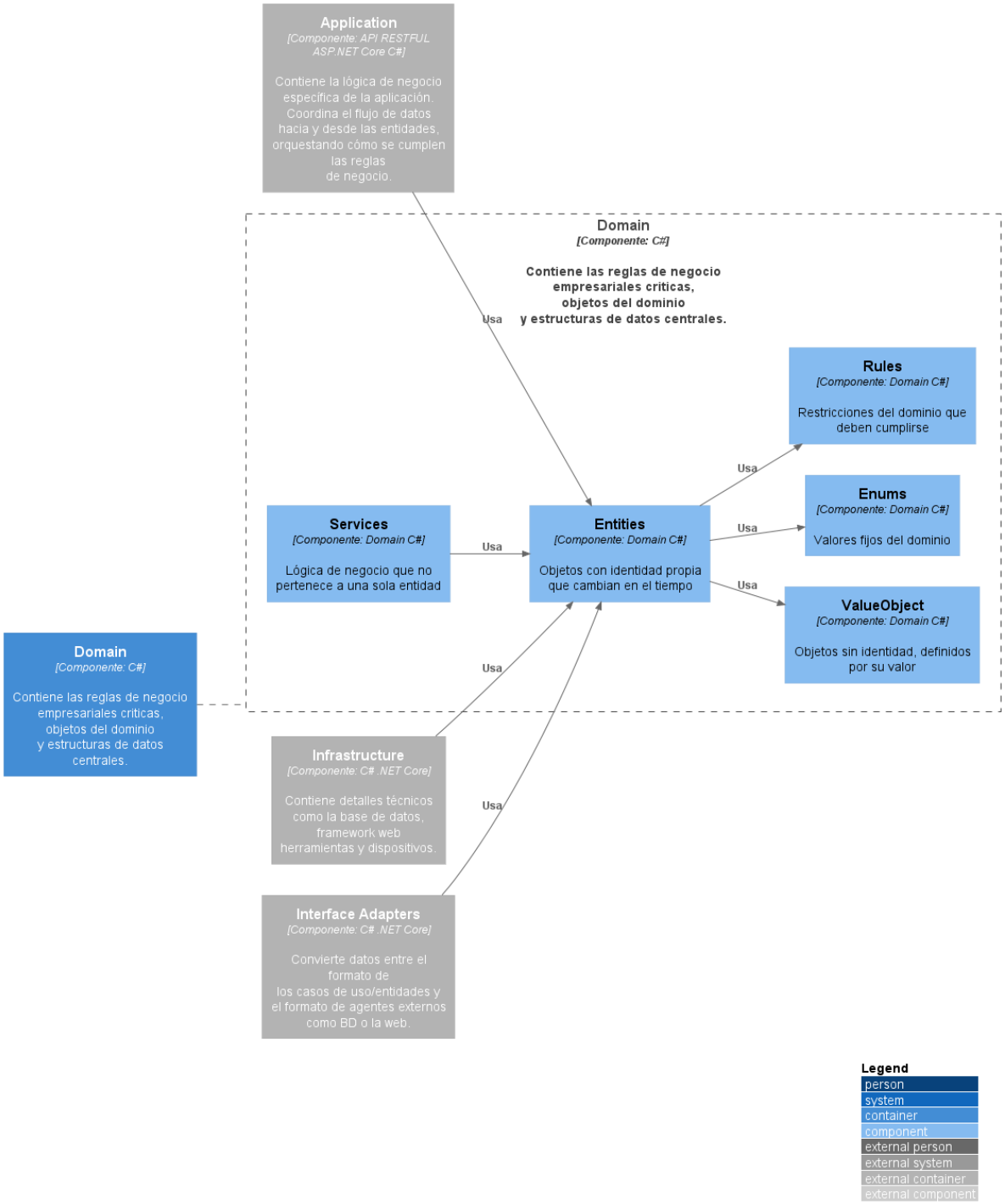

Figure 5: Diagrama C4 - Domain

Responsabilidad

Contiene el **modelo puro del negocio zootécnico**.

No depende de:

- Base de datos
- Frameworks
- HTTP

Entities/

Product.cs

Representa un producto del sistema.

Animal.cs

Entidad principal del dominio.

```
public class Animal
{
    public Guid Id { get; private set; }
    public string Name { get; private set; }

    public void ChangeName(string name)
    {
        if (string.IsNullOrWhiteSpace(name))
            throw new DomainException("Nombre inválido");

        Name = name;
    }
}
```

MilkRecord.cs

Registro de producción de leche.

ReproductionEvent.cs

Eventos reproductivos (celo, fecundación, parto).

AuditEntry.cs

Registro de auditoría del dominio.

ValueObjects/

Objetos sin identidad.

AnimalId.cs

Encapsula el identificador.

Weight.cs

Valida peso.

```
public class Weight
{
    public decimal Value { get; }

    public Weight(decimal value)
    {
        if (value <= 0)
            throw new DomainException("Peso inválido");

        Value = value;
    }
}
```

MilkQuantity.cs

Cantidad de leche con reglas.

Enums/

AnimalStatus.cs

Estados del animal.

ReproductionType.cs

Tipos de reproducción.

Rules/

Contiene reglas de negocio reutilizables.

```
public interface IBusinessRule
{
    bool IsBroken();
    string Message { get; }
}
```

Exceptions/

DomainException.cs

Errores del dominio.

Services/

Servicios de dominio para lógica compleja.

APPLICATION

```
Application/

   Common/
       Gateway/
           Context/
               IRequestContext.cs
               ITenantContext.cs

           Caching/
               IAppCacheService.cs

           Configuration/
               IConfigurationService.cs

           Features/
               IFeatureService.cs

           Rules/
               IRuleEngineService.cs

           Identity/
               ICurrentUserService.cs

           Time/
                IDateTimeProvider.cs

       Behaviors/
       Exceptions/
       Models/

   Modules/
```

```
    Animals/
        UseCases/
            CreateAnimal/
                Ports/
                    ICreateAnimalInputPort.cs
                    ICreateAnimalOutputPort.cs

                CreateAnimalInteractor.cs
                CreateAnimalCommand.cs
                CreateAnimalOutput.cs
                CreateAnimalValidator.cs

        Interfaces/
            IAnimalRepository.cs

        DTOs/

    Reproduction/
        UseCases/
        Interfaces/
            IReproductionRepository.cs

    Production/
        UseCases/
        Interfaces/
            IProductionRepository.cs

    Health/
        UseCases/
        Interfaces/
            IHealthRepository.cs

DependencyInjection.cs
```

Diagrama C4

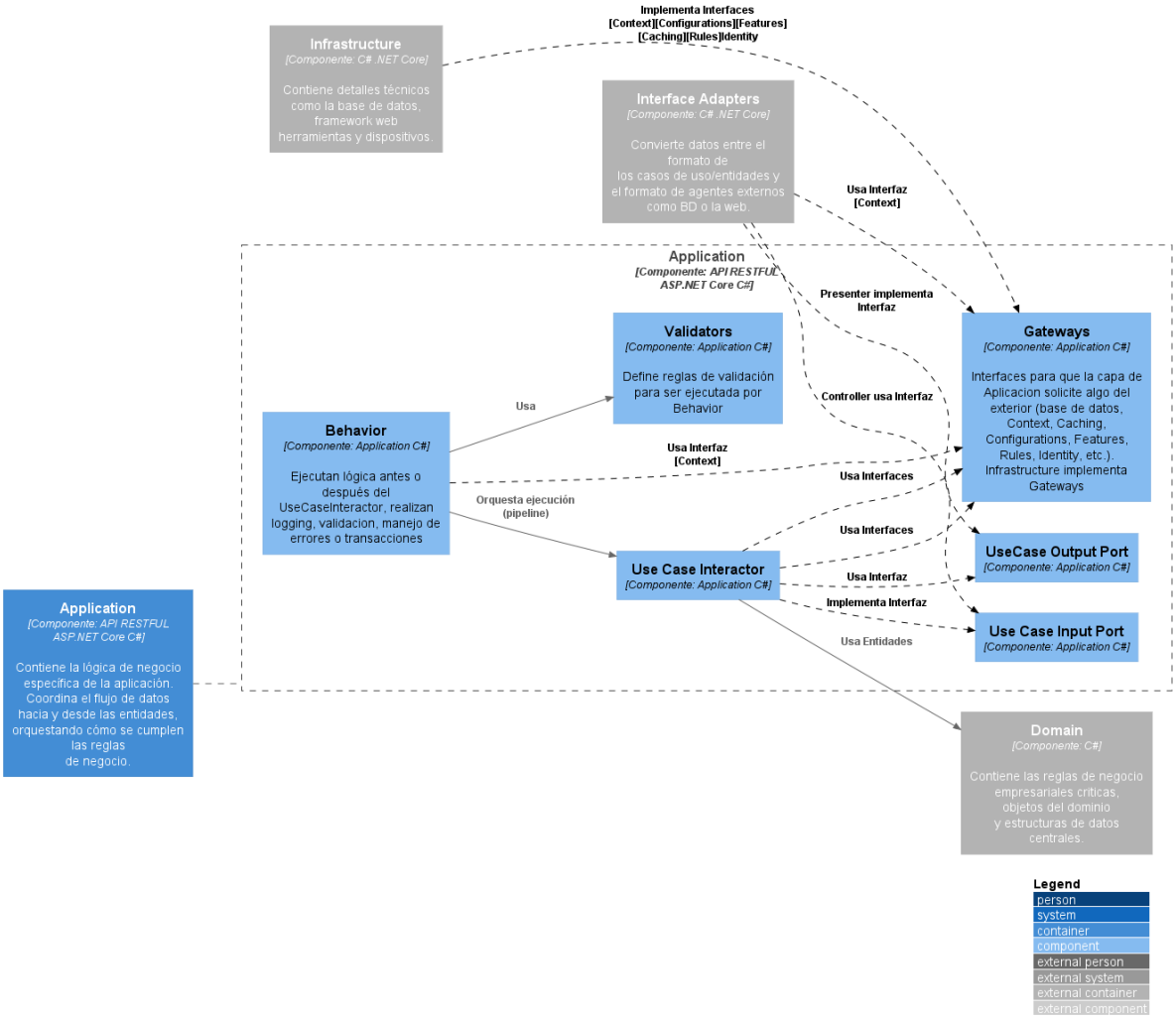

Figure 6: Diagrama C4 - Application

Responsabilidad

Orquesta los casos de uso.

Common/Gateway/

Interfaces para acceso externo.

Context/

- IRequestContext: datos HTTP
- ITenantContext: tenant actual

Caching/

- IAppCacheService

Configuration/

- IConfigurationService

Features/

- IFeatureService

Rules/

- IRuleEngineService

Identity/

- ICurrentUserService

Time/

- IDateTimeProvider

Behaviors/

Pipeline transversal:

- logging
- validación
- auditoría

Modules/

Separación por dominio.

Animals/

Interfaces/

```
public interface IAnimalRepository
{
    Task AddAsync(Animal animal);
}
```

UseCases/CreateAnimal/

Ports/

```
public interface ICreateAnimalInputPort
{
    Task Handle(CreateAnimalCommand command);
}
```

```
public interface ICreateAnimalOutputPort
{
    Task Ok(CreateAnimalOutput output);
}
```

Interactor

```
public class CreateAnimalInteractor : ICreateAnimalInputPort
{
    private readonly IAnimalRepository _repo;

    public async Task Handle(CreateAnimalCommand cmd)
    {
        var animal = new Animal(...);
        await _repo.AddAsync(animal);
    }
}
```

INTERFACE ADAPTERS

```
InterfaceAdapters/
    Controllers/
          ProductsController.cs
          AnimalsController.cs
          ReportsController.cs

    DTOs/
          Requests/
                CreateProductRequest.cs
                RegisterAnimalRequest.cs

          Responses/
                ProductResponse.cs
                AnimalResponse.cs

    Mappers/
          ProductMapper.cs
          AnimalMapper.cs
```

```
Presenters/
      ProductPresenter.cs
      ReportPresenter.cs

Middleware/
      TenantResolutionMiddleware.cs
      ExceptionHandlingMiddleware.cs
      LoggingMiddleware.cs

Filters/
       ValidationFilter.cs
```

Diagrama C4

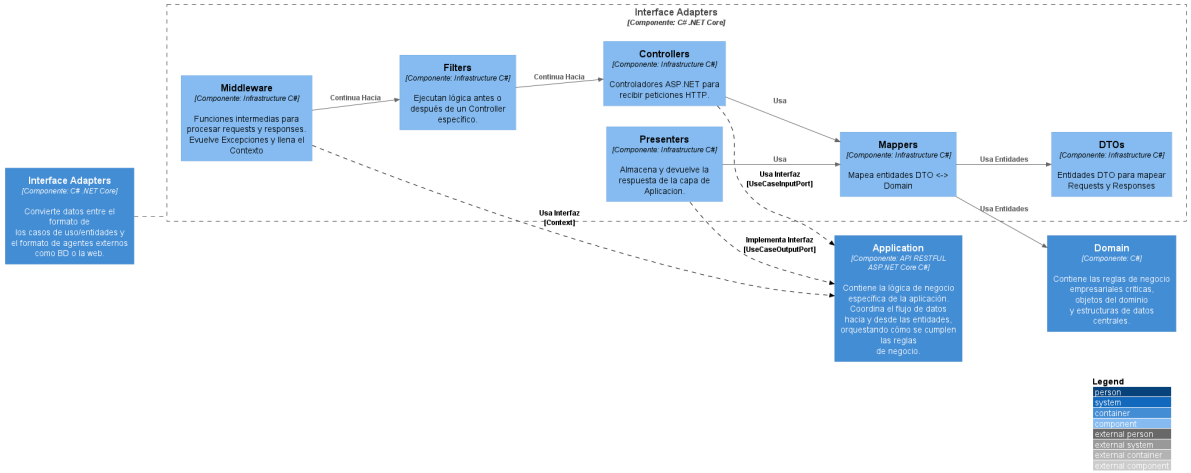

Figure 7: Diagrama C4 - Interface Adapters

Responsabilidad

Adaptar HTTP Application

Controllers/

Reciben requests HTTP.

DTOs/

Objetos de entrada/salida.

Mappers/

Transforman DTO Domain

Presenters/

Formatean respuestas.

Middleware/

TenantResolutionMiddleware

Resuelve tenant por subdominio.

Filters/

Validación automática.

INFRASTRUCTURE

```
Infrastructure/
    Persistence/
          TenantCatalogDb.cs
          GanaderiaDbContext.cs

          Entities/
                ProductEntity.cs
                AnimalEntity.cs
                AppSettingEntity.cs
                FormFieldEntity.cs

          Mappers/
                ProductMapper.cs
                AnimalMapper.cs

          Configurations/
                (Fluent API EF Core)

          Repositories/
                 ProductRepository.cs
                 AnimalRepository.cs

    Tenant/
          TenantContext.cs
          TenantStore.cs
          TenantDbContextFactory.cs
          TenantProvisioningService.cs

    Configuration/
          ConfigurationService.cs

    Features/
          FeatureService.cs

    Rules/
          RuleEngineService.cs

    Caching/
          AppCacheService.cs

    Context/
```

```
        RequestContext.cs

  Identity/
        CurrentUserService.cs

  Auditing/
        AuditSaveChangesInterceptor.cs

  External/
         SenasaClient.cs
         WhatsAppService.cs
         PowerBIExporter.cs
```

Diagrama C4

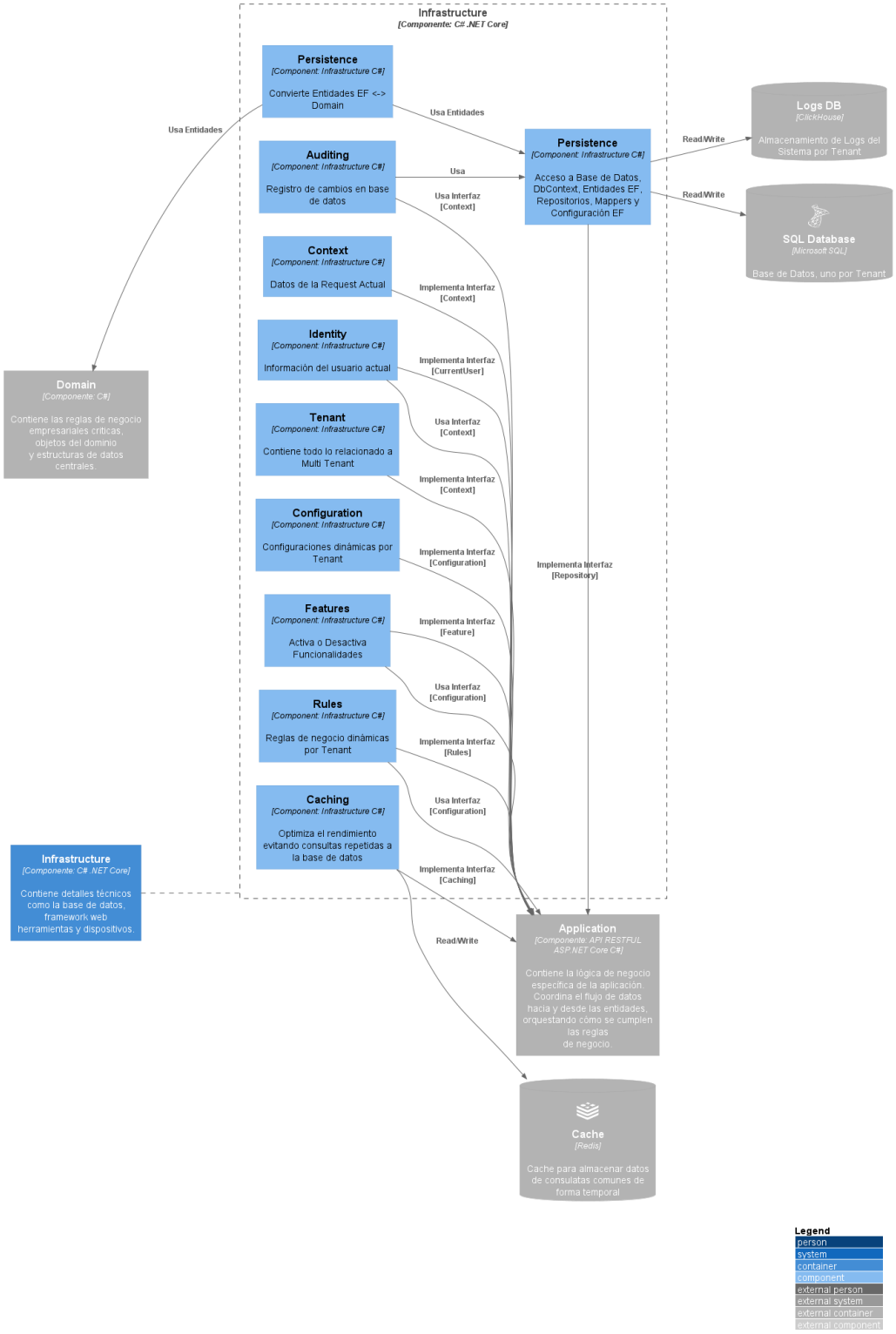

Figure 8: Diagrama C4 - Domain

Responsabilidad

Implementa todo lo técnico.

Persistence/

DbContext

```
public class GanaderiaDbContext : DbContext {}
```

Repositories

Implementan interfaces de Application.

Tenant/

TenantContext

Datos del tenant actual.

TenantStore

Obtiene tenant desde DB.

TenantDbContextFactory

Crea DbContext dinámico.

Configuration/

Lee configuración dinámica.

Features/

Feature flags.

Rules/

Motor de reglas.

Caching/

Cache en memoria.

Context/

Implementa IRequestContext.

Identity/

Usuario actual.

Auditing/

Interceptores EF Core.

External/

Integraciones externas.

FLUJO COMPLETO

```
HTTP Request
 → Middleware (Tenant)
 → Controller
 → UseCase (Application)
 → Domain
 → Repository (Infrastructure)
 → DB
```

CONCLUSIÓN

Arquitectura desacoplada Multi-tenant Altamente configurable Escalable

---